

# Java 8: Streams, map(), default & static Methods

Senior SDET / Java Interview Guide

**Quick memory:** map() = transform | filter() = select | flatMap() = flatten | default = inherited interface implementation | static = interface-level utility method

## 1. Stream map()

**map()** transforms every element of a Stream into another value. It is an intermediate Stream operation and returns a new Stream.

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

List<Integer> squares = numbers.stream()
    .map(n -> n * n)
    .collect(Collectors.toList());

System.out.println(squares);

// Output:
// [1, 4, 9, 16, 25]
```

## 2. map() with String

```
List<String> names = Arrays.asList(
    "swapnil", "amit", "rahul"
);

List<String> upperNames = names.stream()
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(upperNames);

// Output:
// [SWAPNIL, AMIT, RAHUL]
```

## 3. map() to Extract an Object Field

```
class Employee {

    int id;
    String name;

    Employee(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

List<String> names = employees.stream()
    .map(e -> e.name)
    .collect(Collectors.toList());
```

Employee objects are transformed into a Stream of employee names.

## 4. map() vs filter()

| Operation | Purpose | Example |
|-----------|---------|---------|
|-----------|---------|---------|

|           |                          |                                                 |
|-----------|--------------------------|-------------------------------------------------|
| map()     | Transform each element   | $n \rightarrow n * 2$                           |
| filter()  | Select matching elements | $n \rightarrow n > 20$                          |
| flatMap() | Transform + flatten      | $List<List<Integer>> \rightarrow List<Integer>$ |

## 5. map() + filter()

```
List<Integer> numbers =
    Arrays.asList(10, 15, 20, 25, 30);

List<Integer> result = numbers.stream()
    .filter(n -> n > 15)
    .map(n -> n * 2)
    .collect(Collectors.toList());

System.out.println(result);

// Output:
// [40, 50, 60]
```

**Remember:** filter() answers “Which elements do I want?” while map() answers “What should each element become?”

## 6. map() vs flatMap()

```
List<List<Integer>> numbers = Arrays.asList(
    Arrays.asList(1, 2),
    Arrays.asList(3, 4),
    Arrays.asList(5, 6)
);

List<Integer> result = numbers.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());

// Output:
// [1, 2, 3, 4, 5, 6]
```

map() normally produces one result per input element. flatMap() is useful when each input produces a nested collection/stream that needs to be flattened.

## 7. Java 8 default Methods in Interfaces

Java 8 introduced **default methods** so interfaces can contain a method implementation. An implementing class can inherit that implementation or override it.

```
interface Vehicle {

    void start();

    default void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car implements Vehicle {

    @Override
    public void start() {
        System.out.println("Car started");
    }
}

public class Test {

    public static void main(String[] args) {
```

```

        Car car = new Car();

        car.start();
        car.stop();
    }
}

// Output:
// Car started
// Vehicle stopped

```

**Main reason:** default methods allow interfaces to evolve while reducing the need for every existing implementation to immediately provide a new method body.

## 8. Can a default method be overridden?

```

interface Vehicle {

    default void start() {
        System.out.println("Vehicle started");
    }
}

class Car implements Vehicle {

    @Override
    public void start() {
        System.out.println("Car started");
    }
}

```

**Answer:** Yes. A default method can be overridden by the implementing class.

## 9. Java 8 static Methods in Interfaces

An interface can contain a **static method** with an implementation. The method belongs to the interface itself and is called using the interface name.

```

interface Utility {

    static void printMessage() {
        System.out.println("Hello from interface");
    }
}

public class Test {

    public static void main(String[] args) {
        Utility.printMessage();
    }
}

// Output:
// Hello from interface

```

## 10. default vs static

| Feature                     | default              | static |
|-----------------------------|----------------------|--------|
| Has implementation          | Yes                  | Yes    |
| Inherited by implementation | Yes                  | No     |
| Can be overridden           | Yes                  | No     |
| Called using object         | Yes                  | No     |
| Called using interface name | Not the normal usage | Yes    |

## 11. Two Interfaces with the Same default Method

```
interface A {  
  
    default void show() {  
        System.out.println("A");  
    }  
}  
  
interface B {  
  
    default void show() {  
        System.out.println("B");  
    }  
}  
  
class C implements A, B {  
  
    @Override  
    public void show() {  
        System.out.println("C");  
    }  
}
```

If a class implements two interfaces that provide the same default method, the class must resolve the conflict by overriding the method.

### You can explicitly call one interface's default implementation:

```
class C implements A, B {  
  
    @Override  
    public void show() {  
        A.super.show();  
    }  
}
```

## 12. Important Interview Questions

### Q: What is map()?

A: map() transforms each Stream element into another value and returns a new Stream.

### Q: What is filter()?

A: filter() selects elements that satisfy a condition.

### Q: What is flatMap()?

A: flatMap() is used to flatten nested streams/collections after transformation.

### Q: Why were default methods introduced?

A: To allow interfaces to add methods with implementations while improving backward compatibility.

### Q: Can a default method be overridden?

A: Yes.

### Q: Can a static interface method be overridden?

A: No. It belongs to the interface itself.

### Q: How do you call a static interface method?

A: Using InterfaceName.method().

### Q: Can an interface have both default and static methods?

A: Yes, Java 8 supports both.

### Q: What happens when two interfaces have the same default method?

A: The implementing class must resolve the conflict, normally by overriding the method.

## 13. Senior SDET Quick Revision

`map()`

→ Transform every element

`filter()`

→ Select elements based on a condition

`flatMap()`

→ Flatten nested streams

`default`

→ Interface method with implementation

→ Inherited by implementing class

→ Can be overridden

`static`

→ Interface-level method

→ Called with `InterfaceName.method()`

→ Cannot be overridden

## ■ Interview Memory Trick

**map = Modify**

**filter = Find**

**flatMap = Flatten**

**default = Object can use it**

**static = Interface owns it**